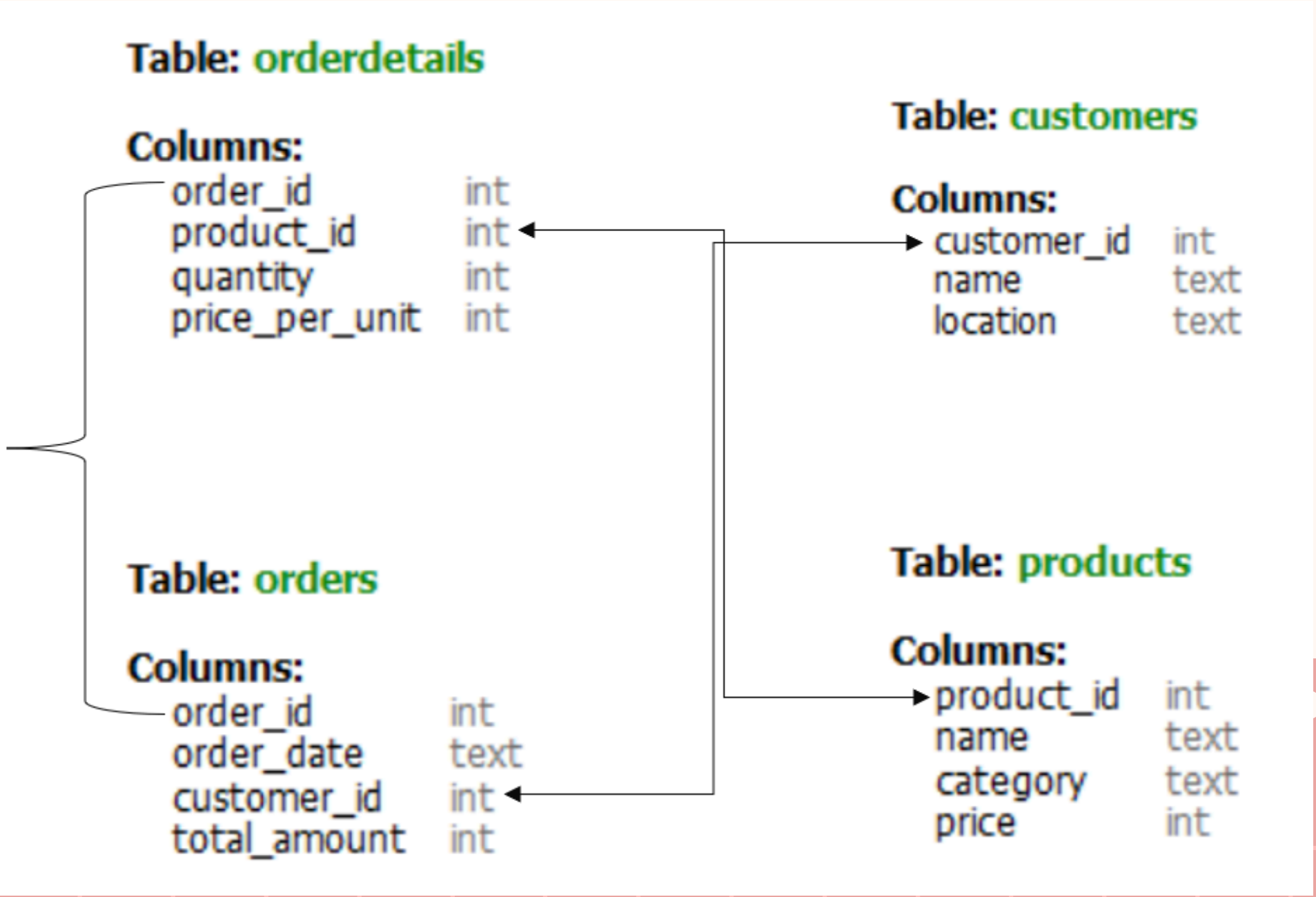
Table Schema



Identify the top 3 cities with the highest number of customers to determine key markets for targeted marketing and logistic optimization.

```sql
SELECT
    location, COUNT(customer_id) AS number_of_customers
FROM
    customers
GROUP BY location
ORDER BY number_of_customers DESC
LIMIT 3;
```

Identify the top 3 cities with the highest number of customers to determine key markets for targeted marketing and logistic optimization.

Result Grid | Filter Rows:

location	number_of_customers
Delhi	16
Chennai	15
Jaipur	11

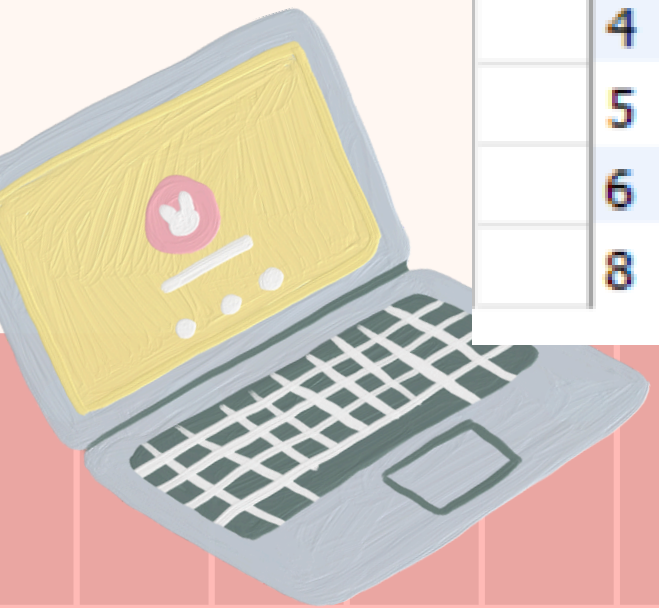
Determine the distribution of customers by the number of orders placed. This insight will help in segmenting customers into one-time buyers, occasional shoppers, and regular customers for tailored marketing strategies.

```sql
SELECT
    NumberOfOrders, COUNT(*) AS CustomerCount
FROM
    (SELECT
        customer_id, COUNT(order_id) AS NumberOfOrders
    FROM
        Orders
    GROUP BY customer_id) AS CustomerOrders
GROUP BY NumberOfOrders
ORDER BY NumberOfOrders ASC;
```

Determine the distribution of customers by the number of orders placed. This insight will help in segmenting customers into one-time buyers, occasional shoppers, and regular customers for tailored marketing strategies.

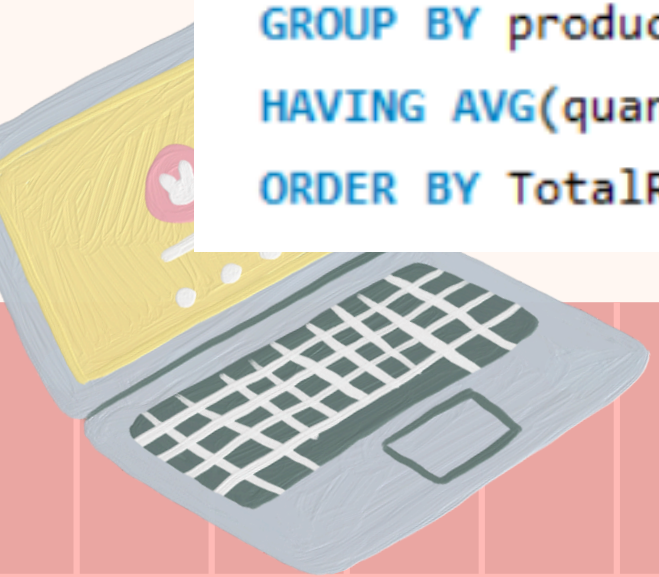
Result Grid | Filter Rows:

NumberOfOrders	CustomerCount
1	26
2	26
3	18
4	6
5	6
6	1
8	1



Identify products where the average purchase quantity per order is 2 but with a high total revenue, suggesting premium product trends.

```sql
SELECT
    product_id,
    AVG(quantity) AS AvgQuantity,
    SUM(quantity * price_per_unit) AS TotalRevenue
FROM
    OrderDetails
GROUP BY product_id
HAVING AVG(quantity) = 2
ORDER BY TotalRevenue DESC;
```



Identify products where the average purchase quantity per order is 2 but with a high total revenue, suggesting premium product trends.

Result Grid | Filter Rows:

	product_id	AvgQuantity	TotalRevenue
▶	1	2.0000	1620000
	8	2.0000	390000

For each product category, calculate the unique number of customers purchasing from it. This will help understand which categories have wider appeal across the customer base.

```sql
SELECT
    p.category,
    COUNT(DISTINCT o.customer_id) AS unique_customers
FROM
    Products p
        JOIN
    OrderDetails od ON p.product_id = od.product_id
        JOIN
    Orders o ON od.order_id = o.order_id
GROUP BY p.category
ORDER BY unique_customers DESC;
```

For each product category, calculate the unique number of customers purchasing from it. This will help understand which categories have wider appeal across the customer base.

Result Grid | Filter Rows:

category	unique_customers
Electronics	79
Wearable Tech	61
Photography	45

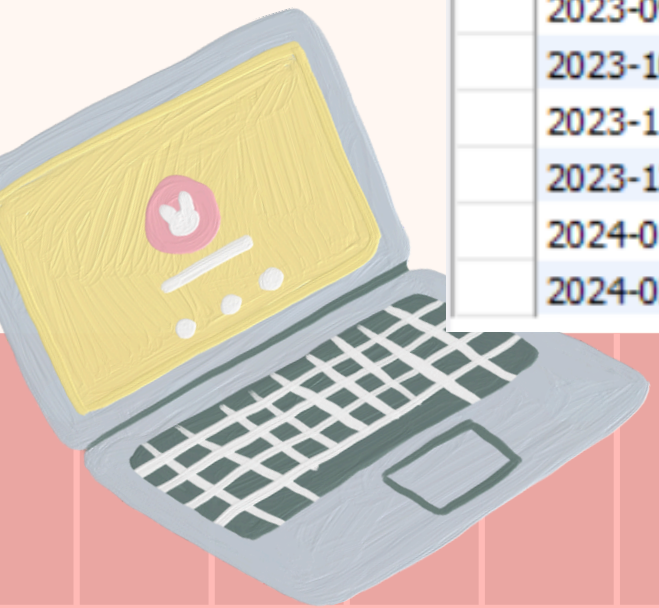
Analyze the month-on-month percentage change in total sales to identify growth trends.

```sql
WITH MonthlySales AS (
    SELECT
        DATE_FORMAT(order_date, '%Y-%m') AS Month,
        SUM(total_amount) AS TotalSales
    FROM Orders
    GROUP BY DATE_FORMAT(order_date, '%Y-%m')
)
SELECT
    Month,
    TotalSales,
    ROUND(
        (
            (TotalSales - LAG(TotalSales) OVER (ORDER BY Month))
            / NULLIF(LAG(TotalSales) OVER (ORDER BY Month), 0)
        ) * 100,
        2
    ) AS PercentChange
FROM MonthlySales
ORDER BY Month;
```

Analyze the month-on-month percentage change in total sales to identify growth trends.

Result Grid | Filter Rows: | Export:

Month	TotalSales	PercentChange
2023-03	789000	NULL
2023-04	1704000	115.97
2023-05	1582000	-7.16
2023-06	1040000	-34.26
2023-07	2568000	146.92
2023-08	1800000	-29.91
2023-09	2927000	62.61
2023-10	1497000	-48.86
2023-11	1151000	-23.11
2023-12	2774000	141.01
2024-01	1555000	-43.94
2024-02	396000	-74.53



Examine how the average order value changes month-on-month. Insights can guide pricing and promotional strategies to enhance order value.

```sql
WITH MonthlyOrderValues AS (
    SELECT
        DATE_FORMAT(order_date, '%Y-%m') AS Month,
        ROUND(AVG(total_amount), 2) AS AvgOrderValue
    FROM Orders
    GROUP BY DATE_FORMAT(order_date, '%Y-%m')
)
SELECT
    Month,
    AvgOrderValue,
    ROUND(
        AvgOrderValue - LAG(AvgOrderValue) OVER (ORDER BY Month),
        2
    ) AS ChangeInValue
FROM MonthlyOrderValues
ORDER BY ChangeInValue DESC;
```

Examine how the average order value changes month-on-month. Insights can guide pricing and promotional strategies to enhance order value.

Result Grid | Filter Rows:

Month	AvgOrderValue	ChangeInValue
2023-12	132095.24	36178.57
2023-04	81142.86	20450.55
2023-06	104000.00	16111.11
2023-08	112500.00	13730.77
2023-11	95916.67	12750.00
2023-09	121958.33	9458.33
2023-05	87888.89	6746.03
2024-01	129583.33	-2511.91
2023-07	98769.23	-5230.77
2023-10	83166.67	-38791.66
2024-02	44000.00	-85583.33
2023-03	60692.31	NULL

Based on sales data, identify products with the fastest turnover rates, suggesting high demand and the need for frequent restocking.

```sql
SELECT
    product_id, COUNT(order_id) AS SalesFrequency
FROM
    OrderDetails
GROUP BY product_id
ORDER BY SalesFrequency DESC
LIMIT 5;
```

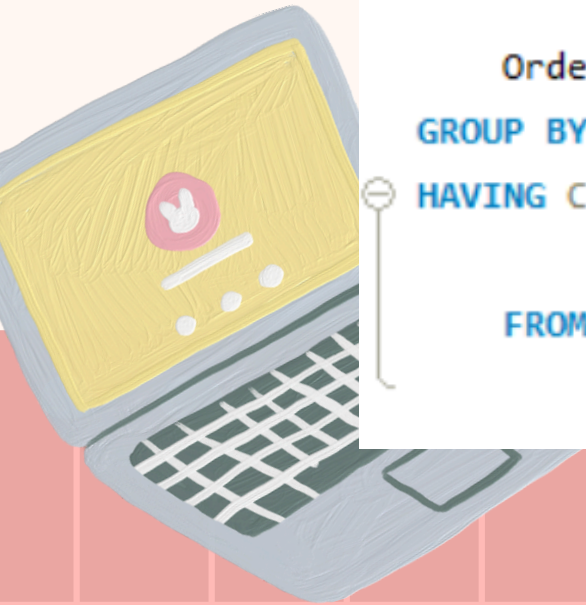
Based on sales data, identify products with the fastest turnover rates, suggesting high demand and the need for frequent restocking.

Result Grid | Filter Rows:

product_id	SalesFrequency
7	78
3	68
4	68
2	67
8	65

List products purchased by less than 40% of the customer base, indicating potential mismatches between inventory and customer interest.

```sql
SELECT
    p.product_id,
    p.name,
    COUNT(DISTINCT o.customer_id) AS UniqueCustomerCount
FROM
    Products p
        JOIN
    OrderDetails od ON p.product_id = od.product_id
        JOIN
    Orders o ON od.order_id = o.order_id
GROUP BY p.product_id , p.name
HAVING COUNT(DISTINCT o.customer_id) < (SELECT
        COUNT(*)
    FROM
        Customers) * 0.40;
```



List products purchased by less than 40% of the customer base, indicating potential mismatches between inventory and customer interest.

Result Grid | Filter Rows: | Ex

product_id	name	UniqueCustomerCount
1	Smartphone 6"	36
8	Wireless Earbuds	38

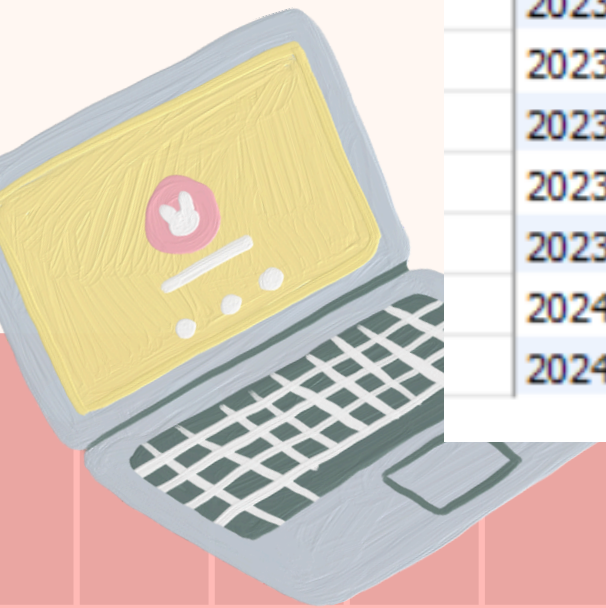
Evaluate the month-on-month growth rate in the customer base to understand the effectiveness of marketing campaigns and market expansion efforts.

```sql
WITH MonthlyNewCustomers AS (
    SELECT
        DATE_FORMAT(MIN(order_date), '%Y-%m') AS FirstPurchaseMonth
    FROM Orders
    GROUP BY customer_id
)
SELECT
    FirstPurchaseMonth,
    COUNT(*) AS TotalNewCustomers
FROM MonthlyNewCustomers
GROUP BY FirstPurchaseMonth
ORDER BY FirstPurchaseMonth;
```

Evaluate the month-on-month growth rate in the customer base to understand the effectiveness of marketing campaigns and market expansion efforts.

Result Grid | Filter Rows: | Export:

FirstPurchaseMonth	TotalNewCustomers
2023-03	11
2023-04	18
2023-05	11
2023-06	8
2023-07	11
2023-08	9
2023-09	5
2023-10	3
2023-11	1
2023-12	4
2024-01	2
2024-02	1



Identify the months with the highest sales volume, aiding in planning for stock levels, marketing efforts, and staffing in anticipation of peak demand periods.

```sql
SELECT
    DATE_FORMAT(order_date, '%Y-%m') AS Month,
    SUM(total_amount) AS TotalSales
FROM
    Orders
WHERE
    order_date IS NOT NULL
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY TotalSales DESC
LIMIT 3;
```

Identify the months with the highest sales volume, aiding in planning for stock levels, marketing efforts, and staffing in anticipation of peak demand periods.

Result Grid | Filter Rows:

Month	TotalSales
2023-09	2927000
2023-12	2774000
2023-07	2568000